

# Zarządzanie maszynami wirtualnymi i sieciami je łączącymi na systemie NixOS z wykorzystaniem języka Nix

Raport projektowy — WSO 2026L

Filip Kobierski (348591)

Andrzej Pultyn (325213)

2 czerwca 2026

## Streszczenie

Celem projektu było opracowanie zestawu narzędzi automatyzujących zarządzanie maszynami wirtualnymi (VM) oraz *sieciami je łączącymi*. Jako scenariusz referencyjny przyjęto trójwarstwową usługę WWW (serwer WWW, pamięć podręczna Redis, baza danych PostgreSQL) wdrożoną jako tzw. *daisy chain* — łańcuch maszyn, w którym każde połączenie między warstwami stanowi odrębny segment sieci. Całość opisano deklaratywnie w języku Nix na systemie NixOS, z wykorzystaniem KVM/QEMU oraz libvirt. Rozwiązanie zapewnia tworzenie i usuwanie sieci oraz VM, filtrację ruchu (nftables, polityka *default-deny*), NAT na brzegu sieci oraz konfigurację zapory dla każdej maszyny. Poprawność i właściwości bezpieczeństwa potwierdzono zautomatyzowanym testem integracyjnym realizującym plan testów (widoczność ICMP, skan portów *nmap*). W raporcie przeanalizowano bezpieczeństwo rozwiązania, wskazując zarówno wzorce bezpieczne, jak i typowe nadużycia.

## Spis treści

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Scenariusz i zakres</b>                          | <b>2</b> |
| <b>2</b> | <b>Użyte narzędzia</b>                              | <b>2</b> |
| <b>3</b> | <b>Architektura</b>                                 | <b>2</b> |
| 3.1      | Topologia „daisy chain” . . . . .                   | 2        |
| 3.2      | Adresacja i macierz dostępu . . . . .               | 2        |
| <b>4</b> | <b>Implementacja</b>                                | <b>2</b> |
| 4.1      | Jedno źródło prawdy: model topologii . . . . .      | 2        |
| 4.2      | Filtracja ruchu i NAT (nftables) . . . . .          | 3        |
| 4.3      | Zarządzanie cyklem życia (vmctl) . . . . .          | 4        |
| <b>5</b> | <b>Plan testów i wyniki</b>                         | <b>4</b> |
| <b>6</b> | <b>Analiza bezpieczeństwa</b>                       | <b>4</b> |
| 6.1      | Zastosowane wzorce bezpieczne . . . . .             | 4        |
| 6.2      | Niewłaściwe sposoby użycia (czego unikać) . . . . . | 5        |
| <b>7</b> | <b>Ryzyko i ograniczenia</b>                        | <b>5</b> |
| <b>8</b> | <b>Wnioski</b>                                      | <b>5</b> |

## 1 Scenariusz i zakres

Zgodnie z tematem projektu zakres obejmuje *zarządzanie sieciami dla maszyn wirtualnych*: tworzenie i usuwanie sieci oraz VM, filtrację ruchu, NAT i konfigurację zapory. Scenariusz z koncepcji to działająca usługa złożona z trzech elementów:

- serwisu WWW (publiczny punkt wejścia),
- pamięci podręcznej Redis,
- bazy danych ze zwykłymi danymi (PostgreSQL).

Maszyny tworzą *daisy chain*: `gateway` → `www` → `cache` → `db`. Struktura ta może w przyszłości ewoluować do drzewa (np. przez dodanie osobnej, ściślej izolowanej bazy z danymi krytycznymi) bez zmiany założeń projektu — wystarczy rozszerzyć model topologii.

## 2 Użyte narzędzia

- **Linux + KVM** — wirtualizacja sprzętowa,
- **libvirt / QEMU** — definiowanie sieci i domen (VM),
- **Nix** — język opisu konfiguracji i budowania artefaktów,
- **NixOS** — deklaratywny system operacyjny każdej VM,
- **nftables** — filtracja ruchu i NAT,
- **nginx, Redis, PostgreSQL** — usługi poszczególnych warstw.

Serwerem docelowym jest maszyna z systemem NixOS (<http://beka.ovh>).

## 3 Architektura

### 3.1 Topologia „daisy chain”

Kluczową decyzją projektową jest umieszczenie *każdego* połączenia między warstwami w osobnym segmencie warstwy 2 (rys. 1). Dzięki temu nie istnieje wspólna domena rozgłoszeniowa — warstwa może komunikować się z inną tylko wtedy, gdy jest fizycznie podłączona do tego samego segmentu *oraz* zapora po drugiej stronie na to zezwala.



Rysunek 1: Łańcuch maszyn i segmenty sieci je łączące. Klient reprezentuje dowolny host w Internecie (tylko w teście).

### 3.2 Adresacja i macierz dostępu

Tabela 1 przedstawia segmenty i adresy, a tabela 2 — jedyne dozwolone przepływy. Wszystko poza nimi jest odrzucane (*default-deny*).

## 4 Implementacja

### 4.1 Jedno źródło prawdy: model topologii

Całe rozwiązanie wyprowadzane jest z jednej struktury danych (`lib/topology.nix`). Konfiguracja sieci hostów, reguły nftables, test integracyjny oraz skrypt zarządzający korzystają z tych samych danych, więc nie mogą się rozjechać. Listing 1 pokazuje fragment opisu hostów.

| segment | CIDR         | VLAN | uczestnicy                    |
|---------|--------------|------|-------------------------------|
| wan     | 192.0.2.0/24 | 9    | gateway .1, client .10 (test) |
| edge    | 10.10.0.0/24 | 1    | gateway .1, www .10           |
| app     | 10.20.0.0/24 | 2    | www .1, cache .10             |
| data    | 10.30.0.0/24 | 3    | cache .1, db .10              |

Tabela 1: Segmenty sieci i adresacja.

| źródło           | cel             | port    | uzasadnienie                     |
|------------------|-----------------|---------|----------------------------------|
| Internet         | gateway (publ.) | 80, 443 | opublikowany serwis (DNAT → www) |
| www              | cache           | 6379    | dostęp do Redis                  |
| cache            | db              | 5432    | dostęp do PostgreSQL             |
| zakres admin.    | każdy host      | 22      | zarządzanie (SSH)                |
| <i>pozostałe</i> | —               | —       | <b>odrzucać</b>                  |

Tabela 2: Macierz dozwolonego ruchu.

Listing 1: Fragment modelu topologii (lib/topology.nix).

```

1 hosts = {
2   gateway = { role = "gateway";
3     nets = [ { net = "wan"; address = "192.0.2.1"; }
4       { net = "edge"; address = "10.10.0.1"; } ]; };
5   www = { role = "www";
6     nets = [ { net = "edge"; address = "10.10.0.10"; }
7       { net = "app"; address = "10.20.0.1"; } ];
8     defaultGateway = "10.10.0.1"; };
9   cache = { role = "cache";
10     nets = [ { net = "app"; address = "10.20.0.10"; }
11       { net = "data"; address = "10.30.0.1"; } ]; };
12   db = { role = "db";
13     nets = [ { net = "data"; address = "10.30.0.10"; } ]; };
14 };

```

Lista `nets` jest *uporządkowana*: kolejność wpisów wyznacza kolejność interfejsów sieciowych maszyny. Pozwala to deterministycznie przypisać sieć logiczną do nazwy interfejsu zarówno w teście NixOS, jak i na realnym hoście libvirt. Funkcja `lib/mkHost.nix` zamienia jeden wpis hosta w kompletny moduł NixOS (adresacja, trasy, zaporę, usługi roli).

## 4.2 Filtracja ruchu i NAT (nftables)

Zamiast nadbudowywać reguły nad wbudowaną zaporą NixOS, generujemy *kompletny*, samodzielny zestaw reguł nftables dla każdej roli (`lib/firewall.nix`). W projekcie nastawionym na bezpieczeństwo wartością jest to, że każdy zaakceptowany pakiet daje się wytłumaczyć na podstawie jednego, audytowalnego dokumentu. Listing 2 pokazuje regułę warstwy bazy danych: ruch przychodzący dozwolony wyłącznie z warstwy cache, a ruch *wychodzący* domyślnie zabroniony (brak możliwości eksfiltracji).

Listing 2: Reguły nftables dla warstwy db (wygenerowane).

```

1 table inet filter {
2   chain input {
3     type filter hook input priority 0; policy drop;
4     iif "lo" accept
5     ct state established,related accept
6     ct state invalid drop
7     ip protocol icmp icmp type echo-request limit rate 5/second accept

```

```

8   ip saddr 10.0.0.0/8 tcp dport 22 accept
9   ip saddr 10.30.0.1 tcp dport 5432 accept # tylko z cache
10  }
11  chain forward { type filter hook forward priority 0; policy drop; }
12  chain output {
13      type filter hook output priority 0; policy drop; # brak egress
14      oif "lo" accept
15      ct state established,related accept
16  }
17  }

```

Na bramie (gateway) dodatkowo działa tablica `nat`: DNAT publikuje serwis WWW na adresie publicznym, a masquerade umożliwia wybranym hostom wewnętrznym (warstwie WWW) wyjście do Internetu. Tylko brama ma włączone przekazywanie pakietów (`ip_forward`); pozostałe maszyny są punktami końcowymi, nie routerami. Warstwy `cache` i `db` nie mają trasy domyślnej, więc nie są w stanie połączyć się z Internetem.

### 4.3 Zarządzanie cyklem życia (vmctl)

Skrypt `scripts/vmctl.sh` stanowi imperatywne uzupełnienie deklaratywnej konfiguracji: NixOS decyduje, co znajduje się *wewnątrz* maszyn, a `vmctl` tworzy i usuwa *sieci* oraz *domeny* libvirt, na których te maszyny działają. Segmenty wewnętrzne to sieci izolowane (bez DHCP i bez routingu hosta — adresacją zarządza NixOS), a segment `wan` to sieć NAT zapewniająca bramie dostęp do Internetu. Najważniejsze podkomendy: `up`, `down`, `net-up`, `vm-up`, `status`, `console`.

## 5 Plan testów i wyniki

Test integracyjny (`tests/integration.nix`) uruchamia cały łańcuch w QEMU — z tych samych modułów, co wdrożenie produkcyjne — i realizuje plan testów z koncepcji, dodając przypadki negatywne, które nadają mu sens:

- **Widoczność ICMP:** `www` pinguje `cache`, `cache` pinguje `db` (warstwy sąsiednie); `client` *nie* pinguje żadnego hosta wewnętrznego; `db` *nie* inicjuje ruchu (brak egress).
- **Skan nmap:** z Internetu widoczne są wyłącznie porty 80/443 na adresie publicznym bramy; porty 22, 5432, 6379 są niewidoczne.
- **Osiągalność:** serwis odpowiada przez adres publiczny (DNAT); dozwolone są `www→cache:6379` i `cache→db:5432`; zablokowane `www→db:5432`, `client→cache:6379`, `client→db:5432`.

Test uruchamia się poleceniem `nix flake check` (lub `nix build .#checks.x86_64-linux.integration-L`) i nie wymaga ręcznej konfiguracji libvirt.

## 6 Analiza bezpieczeństwa

### 6.1 Zastosowane wzorce bezpieczne

**Segmentacja i least privilege.** Jeden segment L2 na połączenie, polityka *default-deny*, usługi udostępniane wyłącznie temu sąsiadowi, który ich potrzebuje — z dopasowaniem po *adresie źródłowym*, nie tylko porcie. Atakujący, który przejmie warstwę WWW, nie ma drogi sieciowej do bazy z pominięciem warstwy `cache`.

**Obrona w głąb (defense in depth).** Filtracja sieciowa *oraz* zabezpieczenia na poziomie usług: Redis nasłuchuje tylko na interfejsie od strony `app` i wymaga hasła; PostgreSQL nasłuchuje tylko na interfejsie od strony `data`, a `pg_hba` ogranicza klientów i wymusza `scram-sha-256`.

**Minimalizacja powierzchni ataku.** Brama wystawiona do Internetu nie uruchamia żadnej usługi aplikacyjnej. Baza danych nie ma egress ani trasy domyślnej, co minimalizuje skutki ewentualnego przejścia (brak „call home”, utrudniona eksfiltracja). Wyłączono dokumentację i zbędne pakiety.

**Reprodukowalność jako bezpieczeństwo.** Cała konfiguracja jest deklaratywna i wersjonowana, co eliminuje dryf konfiguracji i pozwala audytować dopuszczony ruch z jednego modelu. Atomowe aktualizacje i rollback NixOS ograniczają ryzyko nieudanych zmian.

**Twardnienie dostępu.** SSH wyłącznie po kluczu (bez haseł, bez logowania root), a zapora dodatkowo ogranicza zakres adresów źródłowych mogących w ogóle osiągnąć portu 22.

## 6.2 Niewłaściwe sposoby użycia (czego unikać)

**Wystawienie Redis/PostgreSQL na adresie „wildcard”.** Redis domyślnie nie ma uwierzytelniania, a jego `protected-mode` łatwo obejść po powiązaniu z adresem routowalnym — otwarty, bezhasłowy Redis to jeden z najczęstszych realnych incydentów. Dlatego stosujemy powiązanie z jednym interfejsem i hasło i zaporę.

**Traktowanie NAT jak zapory.** NAT ukrywa topologię, ale nie filtruje ruchu w obrębie sieci wewnętrznej. Bezpieczeństwo zapewnia tu nftables, nie sam NAT.

**Zbyt szeroki zakres SSH.** Rozszerzenie `adminCidr` do `0.0.0.0/0` wystawia panel zarządzania na cały Internet.

**Sekrety w repozytorium.** Demonstracyjne `requirePass` oraz poświadczenia bazy są wyłącznie zaślepkami; w produkcji należy dostarczyć je poza repozytorium (np. `requirePassFile`, `agenix/sops`).

**Splaszczanie segmentacji.** Połączenie wszystkich warstw w jedną sieć „dla wygody” niweczy najważniejszą właściwość bezpieczeństwa — baza staje się osiągalna bezpośrednio z warstwy WWW.

**Nadmierny egress.** Pozostawienie warstwie WWW nieograniczonego wyjścia do Internetu (tu: dopuszczone na potrzeby aktualizacji) zwiększa ryzyko eksfiltracji; w środowisku produkcyjnym warto ograniczyć je do konkretnych celów lub serwera lustrzanego.

## 7 Ryzyko i ograniczenia

Zgodnie z koncepcją głównym ryzykiem była praca z niszowym oprogramowaniem (ograniczona dokumentacja, mała społeczność). W praktyce najwięcej uwagi wymagało spójne mapowanie nazw interfejsów: sterownik testów NixOS numeruje interfejsy pozycyjnie od `eth1`, podczas gdy realne maszyny libvirt (z parametrem `net.ifnames=0`) numerują od `eth0`. Rozwiązano to parametrem `baseIndex` w `mkHost` oraz dopasowywaniem reguł zapory po adresach, a nie nazwach interfejsów. Certyfikat TLS warstwy WWW jest samopodpisany (demonstracyjnie); w produkcji należy użyć ACME lub certyfikatu dostarczonego.

## 8 Wnioski

Zrealizowano kompletny, deklaratywny zestaw narzędzi do zarządzania maszynami wirtualnymi i sieciami je łączącymi: tworzenie i usuwanie sieci oraz VM, filtrację ruchu, NAT i konfigurację zapory dla każdej maszyny. Podejście oparte na języku Nix dało jedno źródło prawdy, z którego wyprowadzane są zarówno wdrożenie, jak i automatyczny test potwierdzający właściwości bezpieczeństwa. Architektura *daisy chain* z odrębnymi segmentami, polityką *default-deny*

i obroną w głąb realizuje zasadę najmniejszych uprawnień i jest gotowa do rozwoju w strukturę drzewiastą.